医生模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 医生实体表(doctor)

```
CREATE TABLE `doctor` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '医生ID',
  `user_id` bigint NOT NULL COMMENT '用户ID',
  `doctor_no` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT
NULL COMMENT '医生编号',
  `name` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL
COMMENT '姓名',
  `department_id` bigint NOT NULL COMMENT '科室ID',
  `title` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '职称',
  `expertise` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL COMMENT
'专长',
  `introduction` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
COMMENT '简介',
  `status` tinyint NULL DEFAULT 1 COMMENT '状态(0:离职,1:在职)',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `uk_doctor_no`(`doctor_no` ASC) USING BTREE,
  INDEX `idx_user_id`(`user_id` ASC) USING BTREE,
  INDEX `idx_department_id`(`department_id` ASC) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 1 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_0900_ai_ci COMMENT = '医生表' ROW_FORMAT = Dynamic;
```

1.1.2 医生实体类

```
@Data
@TableName("doctor")
@Schema(description = "医生实体类")
public class Doctor {
    @TableId(type = IdType.AUTO)
    @Schema(description = "医生ID")
    private Long id;

    @Schema(description = "医生编号")
    private String doctorNo;

    @NotBlank(message = "医生姓名不能为空")
    @Size(min = 2, max = 50, message = "医生姓名长度必须在2到50个字符之间")
    @Schema(description = "医生姓名")
    private String name;

    @NotNull(message = "所属科室不能为空")
    @Schema(description = "所属科室ID")
```

半夜撕代码版权所有

```java
    private Long departmentId;

    @Schema(description = "职称")
    private String title;

    @Schema(description = "专长")
    private String expertise;

    @Schema(description = "简介")
    private String introduction;

    @Schema(description = "关联用户ID")
    private Long userId;

    @Schema(description = "状态: 1-在职, 0-离职")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "科室信息")
    private Department department;

    @TableField(exist = false)
    @Schema(description = "用户信息")
    private User user;
}
```

1.2 核心属性说明

- **id**: 医生的唯一标识符，自增主键
- **doctor_no**: 医生编号，唯一，格式为"D"加上年月日和4位序号
- **name**: 医生姓名，必填，长度2-50个字符
- **department_id**: 所属科室ID，必填，关联到department表
- **title**: 医生职称，如主任医师、副主任医师、主治医师等
- **expertise**: 医生专长，文本类型，可以详细描述擅长的疾病和治疗方法
- **introduction**: 医生简介，文本类型，可以包含教育背景、工作经历等
- **user_id**: 关联的用户ID，关联到user表，一个医生对应一个用户账号
- **status**: 医生状态，1表示在职，0表示离职
- **create_time**: 创建时间，自动设置为当前时间
- **update_time**: 更新时间，自动更新为最后修改时间
- **department**: 科室信息，非数据库字段，通过关联查询获取
- **user**: 用户信息，非数据库字段，通过关联查询获取

半夜撕代码版权所有

二、各功能详细讲解

2.1 医生信息管理功能

2.1.1 功能概述

医生信息管理功能允许管理员对医生信息进行全面管理，包括新增医生、编辑医生信息、查看医生详情、删除医生以及更改医生状态等操作。同时，系统支持将医生与用户账号进行绑定，使医生可以登录系统进行相关操作。

2.1.2 详细讲解功能实现流程

1. **新增医生流程**：
   - 前端在`vue3/src/views/backend/doctor/index.vue`中点击"新增医生"按钮，弹出新增表单
   - 填写医生信息并提交，调用`submitForm`方法
   - 如果选择创建新用户，则调用`request.post('/doctor/with-user', {...})`向后端发送请求
   - 如果选择现有用户，则调用`request.post('/doctor', doctorForm)`向后端发送请求
   - 后端`DoctorController`的`createDoctor`或`createDoctorWithUser`方法接收请求
   - 调用`DoctorService`的相应方法处理业务逻辑
   - 生成医生编号，检查科室和用户是否存在，保存医生信息到数据库
   - 返回成功结果给前端，前端刷新医生列表
2. **编辑医生信息流程**：
   - 前端点击"编辑"按钮，弹出编辑表单，显示当前医生信息
   - 修改信息后提交，调用`request.put('/doctor/${doctorForm.id}', updateData)`
   - 如果修改了关联用户，先调用解绑API，再调用绑定API
   - 后端`DoctorController`的`updateDoctor`方法接收请求
   - 调用`DoctorService.updateDoctor`方法更新医生信息
   - 检查修改的科室和用户是否合法，更新医生信息到数据库
   - 返回成功结果给前端，前端刷新医生列表
3. **查看医生详情流程**：
   - 前端点击"详情"按钮，调用`request.get('/doctor/${row.id}')`获取详情
   - 后端`DoctorController`的`getDoctorById`方法接收请求
   - 调用`DoctorService.getDetailById`方法获取医生详细信息，包括关联的科室和用户信息
   - 返回医生详情给前端，前端显示在详情对话框中
4. **删除医生流程**：
   - 前端点击"删除"按钮，弹出确认对话框
   - 确认后调用`request.delete('/doctor/${row.id}')`向后端发送删除请求
   - 后端`DoctorController`的`deleteDoctor`方法接收请求
   - 调用`DoctorService.deleteDoctor`方法处理删除逻辑
   - 检查医生是否有关联的排班、预约、就诊记录等，如有则不允许删除

半夜撕代码版权所有

- 删除医生信息，返回成功结果给前端，前端刷新医生列表

5. **更新医生状态流程**：
   - 前端调用 `request.put('/doctor/${id}/status', {status: status})` 发送请求
   - 后端 `DoctorController` 的 `updateDoctorStatus` 方法接收请求
   - 调用 `DoctorService.updateDoctorStatus` 方法更新医生状态
   - 返回成功结果给前端，前端更新显示状态
6. **绑定/解绑用户流程**：
   - 绑定：前端调用 `request.put('/doctor/${doctorId}/bind/${userId}')` 发送请求
   - 解绑：前端调用 `request.put('/doctor/${doctorId}/unbind')` 发送请求
   - 后端 `DoctorController` 的相应方法接收请求
   - 调用 `DoctorService` 的 `bindUserToDoctor` 或 `unbindUserFromDoctor` 方法处理绑定/解绑逻辑
   - 检查用户是否已绑定其他医生，更新医生关联的用户ID
   - 返回成功结果给前端，前端更新显示

2.1.3 关键代码讲解

1. **医生创建代码**
   【springboot/src/main/java/org/example/springboot/service/DoctorService.java：createDoctor方法】

```java
@Transactional
public Doctor createDoctor(Doctor doctor) {
    // 检查科室是否存在
    Department department =
departmentMapper.selectById(doctor.getDepartmentId());
    if (department == null) {
        throw new ServiceException("科室不存在");
    }

    // 如果关联了用户，检查用户是否存在
    if (doctor.getUserId() != null) {
        User user = userMapper.selectById(doctor.getUserId());
        if (user == null) {
            throw new ServiceException("关联的用户不存在");
        }

        // 检查该用户是否已经关联了医生
        LambdaQueryWrapper<Doctor> queryWrapper = new LambdaQueryWrapper<>();
        queryWrapper.eq(Doctor::getUserId, doctor.getUserId());
        if (doctorMapper.selectCount(queryWrapper) > 0) {
            throw new ServiceException("该用户已关联其他医生");
        }
    }

    // 生成医生编号
    if (!StringUtils.isNotBlank(doctor.getDoctorNo())) {
        doctor.setDoctorNo(generateDoctorNo());
    }
```

```
    // 设置状态和时间
    if (doctor.getStatus() == null) {
        doctor.setStatus(1); // 默认在职
    }

    LocalDateTime now = LocalDateTime.now();
    doctor.setCreateTime(now);
    doctor.setUpdateTime(now);

    if (doctorMapper.insert(doctor) <= 0) {
        throw new ServiceException("医生添加失败");
    }

    return doctor;
}
```

这段代码实现了创建医生的业务逻辑。首先检查科室是否存在，然后检查关联的用户是否存在且未关联其他医生。接着生成医生编号，设置默认状态和时间，最后将医生信息插入数据库。整个过程使用了 `@Transactional` 注解确保事务的完整性。

2. **医生编号生成代码**
【springboot/src/main/java/org/example/springboot/service/DoctorService.java：generateDoctorNo方法】

```
private String generateDoctorNo() {
    // 生成格式为 "D" + 年月日 + 4位序号的医生编号
    String prefix = "D" + LocalDateTime.now().toString().substring(0,
10).replace("-", "");

    // 查询当天最大序号
    LambdaQueryWrapper<Doctor> queryWrapper = new LambdaQueryWrapper<>();
    queryWrapper.likeRight(Doctor::getDoctorNo, prefix)
                .orderByDesc(Doctor::getDoctorNo)
                .last("limit 1");
    Doctor latestDoctor = doctorMapper.selectOne(queryWrapper);

    int sequence = 1;
    if (latestDoctor != null && latestDoctor.getDoctorNo().length() >=
prefix.length() + 4) {
        String sequenceStr =
latestDoctor.getDoctorNo().substring(prefix.length());
        try {
            sequence = Integer.parseInt(sequenceStr) + 1;
        } catch (NumberFormatException e) {
            // 忽略解析错误
        }
    }

    return prefix + String.format("%04d", sequence);
}
```

这段代码实现了医生编号的自动生成逻辑。编号格式为"D"加上当前日期（年月日）和4位序号。首先构造前缀，然后查询当天最大序号，在此基础上加1作为新序号，最后格式化为4位数字并拼接返回。

半夜撕代码版权所有

3. **医生更新代码**
【springboot/src/main/java/org/example/springboot/service/DoctorService.java：updateDoctor方法】

```
@Transactional
public void updateDoctor(Long id, Doctor doctor) {
    // 检查医生是否存在
    Doctor existingDoctor = doctorMapper.selectById(id);
    if (existingDoctor == null) {
        throw new ServiceException("医生不存在");
    }

    // 如果修改了科室，检查科室是否存在
    if (doctor.getDepartmentId() != null &&
!doctor.getDepartmentId().equals(existingDoctor.getDepartmentId())) {
        Department department =
departmentMapper.selectById(doctor.getDepartmentId());
        if (department == null) {
            throw new ServiceException("科室不存在");
        }
    }

    // 如果修改了关联用户，检查用户是否存在且未关联其他医生
    if (doctor.getUserId() != null &&
!doctor.getUserId().equals(existingDoctor.getUserId())) {
        User user = userMapper.selectById(doctor.getUserId());
        if (user == null) {
            throw new ServiceException("关联的用户不存在");
        }

        // 检查该用户是否已经关联了医生
        LambdaQueryWrapper<Doctor> queryWrapper = new LambdaQueryWrapper<>();
        queryWrapper.eq(Doctor::getUserId, doctor.getUserId())
                    .ne(Doctor::getId, id);
        if (doctorMapper.selectCount(queryWrapper) > 0) {
            throw new ServiceException("该用户已关联其他医生");
        }
    }

    doctor.setId(id);
    doctor.setUpdateTime(LocalDateTime.now());

    if (doctorMapper.updateById(doctor) <= 0) {
        throw new ServiceException("医生信息更新失败");
    }
}
```

这段代码实现了更新医生信息的业务逻辑。首先检查医生是否存在，然后检查修改的科室是否存在，以及修改的关联用户是否合法。最后更新医生信息到数据库。

4. **医生删除代码**
【springboot/src/main/java/org/example/springboot/service/DoctorService.java：deleteDoctor方法】

```
@Transactional
```

半夜撕代码版权所有

```java
public void deleteDoctor(Long id) {
    Doctor doctor = doctorMapper.selectById(id);
    if (doctor == null) {
        throw new ServiceException("医生不存在");
    }

    // 检查医生是否是科室负责人
    LambdaQueryWrapper<Department> departmentQueryWrapper = new
LambdaQueryWrapper<>();
    departmentQueryWrapper.eq(Department::getDirectorId, id);
    long departmentCount =
departmentMapper.selectCount(departmentQueryWrapper);
    if (departmentCount > 0) {
        throw new ServiceException("该医生是科室负责人，不能删除");
    }

    // 检查医生是否有关联排班
    LambdaQueryWrapper<Schedule> scheduleQueryWrapper = new
LambdaQueryWrapper<>();
    scheduleQueryWrapper.eq(Schedule::getDoctorId, id);
    long scheduleCount = scheduleMapper.selectCount(scheduleQueryWrapper);
    if (scheduleCount > 0) {
        throw new ServiceException("该医生已有排班记录，不能删除");
    }

    // 检查医生是否有关联预约
    LambdaQueryWrapper<Appointment> appointmentQueryWrapper = new
LambdaQueryWrapper<>();
    appointmentQueryWrapper.eq(Appointment::getDoctorId, id);
    long appointmentCount =
appointmentMapper.selectCount(appointmentQueryWrapper);
    if (appointmentCount > 0) {
        throw new ServiceException("该医生已有预约记录，不能删除");
    }

    // 检查医生是否有关联就诊记录
    LambdaQueryWrapper<MedicalRecord> medicalRecordQueryWrapper = new
LambdaQueryWrapper<>();
    medicalRecordQueryWrapper.eq(MedicalRecord::getDoctorId, id);
    long medicalRecordCount =
medicalRecordMapper.selectCount(medicalRecordQueryWrapper);
    if (medicalRecordCount > 0) {
        throw new ServiceException("该医生已有就诊记录，不能删除");
    }

    // 检查医生是否有关联处方
    LambdaQueryWrapper<Prescription> prescriptionQueryWrapper = new
LambdaQueryWrapper<>();
    prescriptionQueryWrapper.eq(Prescription::getDoctorId, id);
    long prescriptionCount =
prescriptionMapper.selectCount(prescriptionQueryWrapper);
    if (prescriptionCount > 0) {
        throw new ServiceException("该医生已有处方记录，不能删除");
    }

    // 如果医生已关联用户，先解绑
```

半夜撕代码版权所有

```
    if (doctor.getUserId() != null) {
        User user = userMapper.selectById(doctor.getUserId());
        if (user != null) {
            // 解绑逻辑
        }
    }

    doctorMapper.deleteById(id);
}
```

这段代码实现了删除医生的业务逻辑。首先检查医生是否存在，然后检查医生是否是科室负责人，以及是否有关联的排班、预约、就诊记录和处方。如果有任何关联数据，则不允许删除。最后，如果医生已关联用户，先解绑用户，然后删除医生信息。

5. **前端医生管理代码**

【vue3/src/views/backend/doctor/index.vue：submitForm方法】

```
const submitForm = async () => {
  let doctorValid = false;
  let userValid = true;

  // 验证医生表单
  await doctorFormRef.value.validate((valid) => {
    doctorValid = valid;
  });

  // 如果选择创建新用户，验证新用户表单
  if (userType.value === 'new' && doctorValid) {
    await newUserFormRef.value.validate((valid) => {
      userValid = valid;
    });
  }

  if (!doctorValid || !userValid) {
    return;
  }

  try {
    loading.value = true;

    if (dialogType.value === 'add') {
      // 新增医生
      if (userType.value === 'new') {
        // 创建医生并同时创建用户
        await request.post('/doctor/with-user', {
          user: {
            username: newUserForm.username,
            password: newUserForm.password,
            email: newUserForm.email,
            phone: newUserForm.phone,
            name: doctorForm.name // 用医生姓名作为用户姓名
          },
          doctor: doctorForm
        }, {
          successMsg: '新增医生和用户成功',
          onSuccess: () => {
```

```
            dialogVisible.value = false;
            fetchDoctors();
            fetchUsers(); // 刷新用户列表
          }
        });
      } else {
        // 使用现有用户创建医生
        await request.post('/doctor', doctorForm, {
          successMsg: '新增医生成功',
          onSuccess: () => {
            dialogVisible.value = false;
            fetchDoctors();
          }
        });
      }
    } else {
      // 编辑医生
      // 获取当前医生信息
      const { data: currentDoctor } = await
request.get(`/doctor/${doctorForm.id}`, {}, {
        showDefaultMsg: false
      });

      // 如果用户ID发生变化，处理绑定关系
      if (currentDoctor && currentDoctor.userId !== doctorForm.userId) {
        // 先解绑当前用户
        if (currentDoctor.userId) {
          await request.put(`/doctor/${doctorForm.id}/unbind`, {}, {
            showDefaultMsg: false,
            onError: (error) => {
              ElMessage.error('解绑原关联用户失败: ' + (error.message || '未知错
误'));
              throw new Error('解绑用户失败');
            }
          });
        }

        // 再绑定新用户（如果选择了新用户）
        if (doctorForm.userId) {
          await
request.put(`/doctor/${doctorForm.id}/bind/${doctorForm.userId}`, {}, {
            showDefaultMsg: false,
            onError: (error) => {
              ElMessage.error('绑定新关联用户失败: ' + (error.message || '未知错
误'));
              throw new Error('绑定用户失败');
            }
          });
        }
      }

      // 更新医生基本信息（不包含userId，因为已经通过专门的API修改了）
      const updateData = { ...doctorForm };
      delete updateData.userId; // 移除userId，避免重复更新

      await request.put(`/doctor/${doctorForm.id}`, updateData, {
```

```
        successMsg: '编辑医生成功',
        onSuccess: () => {
          dialogVisible.value = false;
          fetchDoctors();
        },
        onError: (error) => {
          ElMessage.error('更新医生信息失败: ' + (error.message || '未知错误'));
        }
      });
    }
  } catch (error) {
    console.error('操作失败:', error);
    // 错误信息已在各个步骤中处理
  } finally {
    loading.value = false;
  }
}
```

这段代码实现了前端提交医生表单的逻辑。首先验证表单数据的有效性，然后根据操作类型（新增或编辑）和用户类型（新建用户或使用现有用户）执行不同的操作。对于编辑操作，还需要处理用户绑定关系的变化。

2.1.4 功能实现细节

1. **医生编号自动生成**：
   - 系统自动生成医生编号，格式为"D"加上当前日期（年月日）和4位序号
   - 序号从1开始，每天重新计数
   - 确保编号唯一性，通过查询当天最大序号并加1实现
2. **用户关联管理**：
   - 一个医生只能关联一个用户，一个用户也只能关联一个医生
   - 创建医生时可以选择关联现有用户或创建新用户
   - 编辑医生时可以更换关联用户，系统会自动处理解绑和绑定操作
   - 用户角色自动设置为"DOCTOR"
3. **数据完整性检查**：
   - 删除医生前检查是否有关联的排班、预约、就诊记录和处方
   - 如果有关联数据，不允许删除医生，确保数据完整性
   - 检查医生是否是科室负责人，如果是则不允许删除
4. **前端表单验证**：
   - 医生姓名必填，长度2-50个字符
   - 所属科室必选
   - 如果选择关联现有用户，用户ID必填
   - 如果选择创建新用户，用户名、密码、邮箱和手机号必填且符合格式要求
5. **状态管理**：
   - 医生状态分为在职(1)和离职(0)
   - 状态变更通过专门的API接口实现
   - 前端使用不同颜色的标签直观显示状态

半夜撕代码版权所有

2.2 医生前台展示功能

2.2.1 功能概述

医生前台展示功能向患者展示医院的医生团队信息，包括医生列表、医生详情、出诊信息等。患者可以根据科室、姓名、职称等条件筛选医生，查看医生详情，并进行预约挂号操作。

2.2.2 详细讲解功能实现流程

1. **医生列表展示流程**：
   - 前端在 `vue3/src/views/frontend/doctor/index.vue` 中加载页面时调用 `fetchDoctors` 方法
   - 向后端发送 `request.get('/doctor/page', {...})` 请求获取医生列表
   - 后端 `DoctorController` 的 `getDoctorsByPage` 方法接收请求
   - 调用 `DoctorService.getDoctorsByPage` 方法查询医生列表，包含分页和筛选条件
   - 返回医生列表数据给前端，前端渲染医生卡片列表
2. **医生详情展示流程**：
   - 前端点击医生卡片或"查看详情"按钮，调用 `viewDoctorDetail` 方法
   - 将选中的医生信息保存到 `selectedDoctor` 变量
   - 调用 `fetchDoctorSchedules` 方法获取医生排班信息
   - 向后端发送 `request.get('/schedule/doctor/${doctorId}')` 请求
   - 后端返回排班信息，前端在详情页面展示医生基本信息和排班信息
3. **医生搜索筛选流程**：
   - 前端填写搜索表单，点击搜索按钮，调用 `handleSearch` 方法
   - 将搜索条件作为参数，向后端发送请求获取符合条件的医生列表
   - 后端根据条件筛选医生数据并返回
   - 前端更新医生列表显示
4. **预约挂号入口**：
   - 前端在医生卡片或详情页面提供"预约挂号"按钮
   - 用户点击按钮，调用 `goToAppointment` 或 `goToAppointmentWithSchedule` 方法
   - 跳转到预约挂号页面，并传递医生ID和排班ID（如果有）作为参数

2.2.3 关键代码讲解

1. **前台医生列表获取代码**
   【vue3/src/views/frontend/doctor/index.vue：fetchDoctors方法】

```
const fetchDoctors = async () => {
  loading.value = true;
  try {
    await request.get('/doctor/page', {
      name: searchForm.name,
      departmentId: searchForm.departmentId,
      title: searchForm.title,
      status: 1, // 只显示在职医生
      currentPage: currentPage.value,
```

半夜撕代码版权所有

```
        size: pageSize.value
      }, {
        onSuccess: (res) => {
          doctors.value = res.records;
          total.value = res.total;
        }
      });
    } catch (error) {
      console.error('获取医生列表失败:', error);
      ElMessage.error('获取医生列表失败');
    } finally {
      loading.value = false;
    }
  };
```

这段代码实现了前台获取医生列表的逻辑。通过调用后端API获取医生分页数据，可以根据姓名、科室、职称等条件筛选，且只显示在职医生。获取到数据后更新前端显示。

2. **医生排班信息获取代码**
   【vue3/src/views/frontend/doctor/index.vue：fetchDoctorSchedules方法】

```
const fetchDoctorSchedules = async () => {
  if (!selectedDoctor.value || !selectedDoctor.value.id) return;

  schedulesLoading.value = true;
  try {
    await request.get(`/schedule/doctor/${selectedDoctor.value.id}`, {}, {
      onSuccess: (res) => {
        schedules.value = res || [];
      }
    });
  } catch (error) {
    console.error('获取医生排班信息失败:', error);
    ElMessage.error('获取医生排班信息失败');
  } finally {
    schedulesLoading.value = false;
  }
};
```

这段代码实现了获取医生排班信息的逻辑。当用户查看医生详情时，系统会调用后端API获取该医生的排班信息，以便患者了解医生的出诊时间并进行预约。

3. **医生详情查看代码**
   【vue3/src/views/frontend/doctor/index.vue：viewDoctorDetail方法】

```
const viewDoctorDetail = (doctor) => {
  selectedDoctor.value = doctor;
  fetchDoctorSchedules(); // 获取医生排班信息

  // 如果是移动设备，滚动到顶部
  if (window.innerWidth < 768) {
    window.scrollTo({ top: 0, behavior: 'smooth' });
  }
};
```

半夜撕代码版权所有

这段代码实现了查看医生详情的逻辑。当用户点击医生卡片或"查看详情"按钮时，系统会保存选中的医生信息，并获取该医生的排班信息，然后显示详情页面。

4. **预约挂号跳转代码**
   【vue3/src/views/frontend/doctor/index.vue：goToAppointment方法】

```
const goToAppointment = (doctor) => {
  if (!isLoggedIn.value) {
    ElMessageBox.confirm('请先登录后再进行预约', '提示', {
      confirmButtonText: '去登录',
      cancelButtonText: '取消',
      type: 'warning'
    }).then(() => {
      router.push({
        path: '/login',
        query: { redirect: router.currentRoute.value.fullPath }
      });
    }).catch(() => {});
    return;
  }

  router.push({
    path: '/appointment',
    query: { doctorId: doctor.id }
  });
};
```

这段代码实现了预约挂号跳转的逻辑。当用户点击"预约挂号"按钮时，系统会先检查用户是否已登录，如果未登录则提示用户登录，如果已登录则跳转到预约挂号页面，并传递医生ID作为参数。

2.1.4 功能实现细节

1. **医生卡片展示**：
   - 使用卡片形式展示医生信息，包括姓名、职称、科室、专长等
   - 显示医生头像，如果没有则显示默认头像
   - 提供"查看详情"和"预约挂号"按钮
   - 使用不同颜色的标签区分医生职称
2. **医生详情页设计**：
   - 左侧显示医生基本信息，包括头像、姓名、职称、科室等
   - 右侧使用选项卡形式展示医生简介、专业特长、出诊信息和患者评价
   - 出诊信息以日历卡片形式展示，直观显示可预约时间
   - 提供"预约挂号"按钮，方便患者直接预约
3. **排班信息展示**：
   - 以卡片形式展示医生排班信息，包括日期、时间段、预约人数等
   - 使用不同颜色标识排班状态：可预约、即将满员、已满、停诊
   - 显示预约进度条，直观展示剩余号源情况
   - 对于已满或停诊的排班，禁用预约按钮
4. **搜索筛选功能**：

半夜撕代码版权所有

- 提供多条件搜索，包括科室、医生姓名、职称等
- 实时更新搜索结果，支持分页显示
- 提供重置按钮，清空搜索条件

5. **响应式设计**：
   - 适配不同设备屏幕，在移动设备上优化显示效果
   - 在移动设备上查看详情时自动滚动到顶部，提升用户体验

2.3 医生账号管理功能

2.3.1 功能概述

医生账号管理功能允许管理员为医生创建和管理用户账号，使医生可以登录系统进行相关操作。同时，系统支持将医生与用户账号进行绑定和解绑，以及修改医生关联的用户信息。

2.3.2 详细讲解功能实现流程

1. **创建医生账号流程**：
   - 前端在新增医生表单中选择"创建新用户"选项
   - 填写用户信息（用户名、密码、邮箱、手机号）
   - 提交表单，调用 `request.post('/doctor/with-user', {...})` 向后端发送请求
   - 后端 `DoctorController` 的 `createDoctorWithUser` 方法接收请求
   - 解析请求参数，分别获取医生信息和用户信息
   - 调用 `DoctorService.createDoctorWithUser` 方法处理业务逻辑
   - 先创建用户，设置用户角色为"DOCTOR"，然后创建医生并关联用户
   - 返回成功结果给前端
2. **绑定现有用户流程**：
   - 前端在新增或编辑医生表单中选择"选择已有用户"选项
   - 从下拉列表中选择一个用户
   - 提交表单，对于新增医生直接包含用户ID，对于编辑医生则调用绑定API
   - 后端处理绑定逻辑，检查用户是否已绑定其他医生
   - 更新医生关联的用户ID，完成绑定
3. **解绑用户流程**：
   - 前端在编辑医生表单中清除关联用户，或者选择其他用户
   - 提交表单，系统检测到用户ID变化，调用解绑API
   - 后端 `DoctorController` 的 `unbindUserFromDoctor` 方法接收请求
   - 调用 `DoctorService.unbindUserFromDoctor` 方法处理解绑逻辑
   - 将医生关联的用户ID设为null，完成解绑

2.3.3 关键代码讲解

1. **创建医生并同时创建用户代码**
   【springboot/src/main/java/org/example/springboot/service/DoctorService.java：createDoctorWithUser方法】

半夜撕代码版权所有

```java
@Transactional
public Doctor createDoctorWithUser(Doctor doctor, User user) {
    // 先创建用户
    user.setStatus(1); // 设置用户状态为启用
    user.setCreateTime(LocalDateTime.now());
    user.setUpdateTime(LocalDateTime.now());

    // 对密码进行加密处理
    // 这里应该有对密码的加密逻辑，如果没有请使用BCryptPasswordEncoder

    userMapper.insert(user);

    // 将创建的用户ID关联到医生
    doctor.setUserId(user.getId());

    // 创建医生
    return createDoctor(doctor);
}
```

这段代码实现了创建医生并同时创建用户的业务逻辑。首先创建用户账号，设置状态为启用，然后将创建的用户ID关联到医生，最后调用`createDoctor`方法创建医生。整个过程使用了`@Transactional`注解确保事务的完整性。

2. **绑定用户到医生代码**
   【springboot/src/main/java/org/example/springboot/service/DoctorService.java：bindUserToDoctor方法】

```java
@Transactional
public void bindUserToDoctor(Long doctorId, Long userId) {
    Doctor doctor = doctorMapper.selectById(doctorId);
    if (doctor == null) {
        throw new ServiceException("医生不存在");
    }

    User user = userMapper.selectById(userId);
    if (user == null) {
        throw new ServiceException("用户不存在");
    }

    // 检查用户是否已绑定其他医生
    LambdaQueryWrapper<Doctor> queryWrapper = new LambdaQueryWrapper<>();
    queryWrapper.eq(Doctor::getUserId, userId);
    queryWrapper.ne(Doctor::getId, doctorId);
    Long count = doctorMapper.selectCount(queryWrapper);
    if (count > 0) {
        throw new ServiceException("该用户已绑定其他医生");
    }

    // 更新医生关联的用户ID
    Doctor updateDoctor = new Doctor();
    updateDoctor.setId(doctorId);
    updateDoctor.setUserId(userId);
    updateDoctor.setUpdateTime(LocalDateTime.now());
    doctorMapper.updateById(updateDoctor);
```

半夜撕代码版权所有

```
}
```

这段代码实现了绑定用户到医生的业务逻辑。首先检查医生和用户是否存在，然后检查用户是否已绑定其他医生，最后更新医生关联的用户ID完成绑定。

3. **解绑用户代码**
   【springboot/src/main/java/org/example/springboot/service/DoctorService.java：unbindUserFromDoctor方法】

```
@Transactional
public void unbindUserFromDoctor(Long doctorId) {
    Doctor doctor = doctorMapper.selectById(doctorId);
    if (doctor == null) {
        throw new ServiceException("医生不存在");
    }

    Doctor updateDoctor = new Doctor();
    updateDoctor.setId(doctorId);
    updateDoctor.setUserId(null);
    updateDoctor.setUpdateTime(LocalDateTime.now());

    doctorMapper.updateById(updateDoctor);
}
```

这段代码实现了解绑用户的业务逻辑。首先检查医生是否存在，然后将医生关联的用户ID设为null，完成解绑。

4. **前端用户类型切换代码**
   【vue3/src/views/backend/doctor/index.vue：handleUserTypeChange方法】

```
const handleUserTypeChange = () => {
  // 重置用户相关字段
  doctorForm.userId = null;

  // 重置新用户表单
  if (userType.value === 'new') {
    Object.keys(newUserForm).forEach(key => {
      newUserForm[key] = ''
    })
  }
}
```

这段代码实现了前端用户类型切换的逻辑。当用户在"选择已有用户"和"创建新用户"之间切换时，重置相关字段，确保数据的一致性。

2.3.4 功能实现细节

1. **用户角色管理**：
   - 为医生创建的用户账号自动设置角色为"DOCTOR"
   - 只有DOCTOR角色的用户才能在医生管理页面被选择关联
2. **用户唯一性检查**：
   - 一个用户只能关联一个医生，一个医生也只能关联一个用户

半夜撕代码版权所有

    - 绑定用户时检查用户是否已绑定其他医生
    - 如果已绑定，则不允许绑定，确保数据的一致性
3. **用户表单验证**：
    - 创建新用户时，用户名、密码、邮箱和手机号必填且符合格式要求
    - 用户名长度3-50个字符
    - 密码长度6-20个字符
    - 邮箱必须符合邮箱格式
    - 手机号必须符合手机号格式（中国大陆手机号）
4. **用户列表过滤**：
    - 在选择用户下拉列表中，只显示角色为"DOCTOR"且状态为启用的用户
    - 通过API参数过滤，减轻前端负担

三、总结

3.1 医生信息管理实现

医生信息管理模块实现了对医生基本信息的全面管理。系统通过DoctorController接收前端请求，调用DoctorService处理业务逻辑，使用DoctorMapper与数据库交互。创建医生时，系统会自动生成格式为"D"+年月日+4位序号的医生编号，并验证科室存在性和用户关联合法性。编辑医生信息时，系统会检查修改的科室和用户关联是否合法，确保数据一致性。删除医生前，系统会检查医生是否有关联的排班、预约、就诊记录和处方，以及是否是科室负责人，只有没有关联数据的医生才能被删除，保证数据完整性。前端采用表格形式展示医生列表，支持分页和多条件搜索，提供新增、编辑、详情和删除等操作按钮。

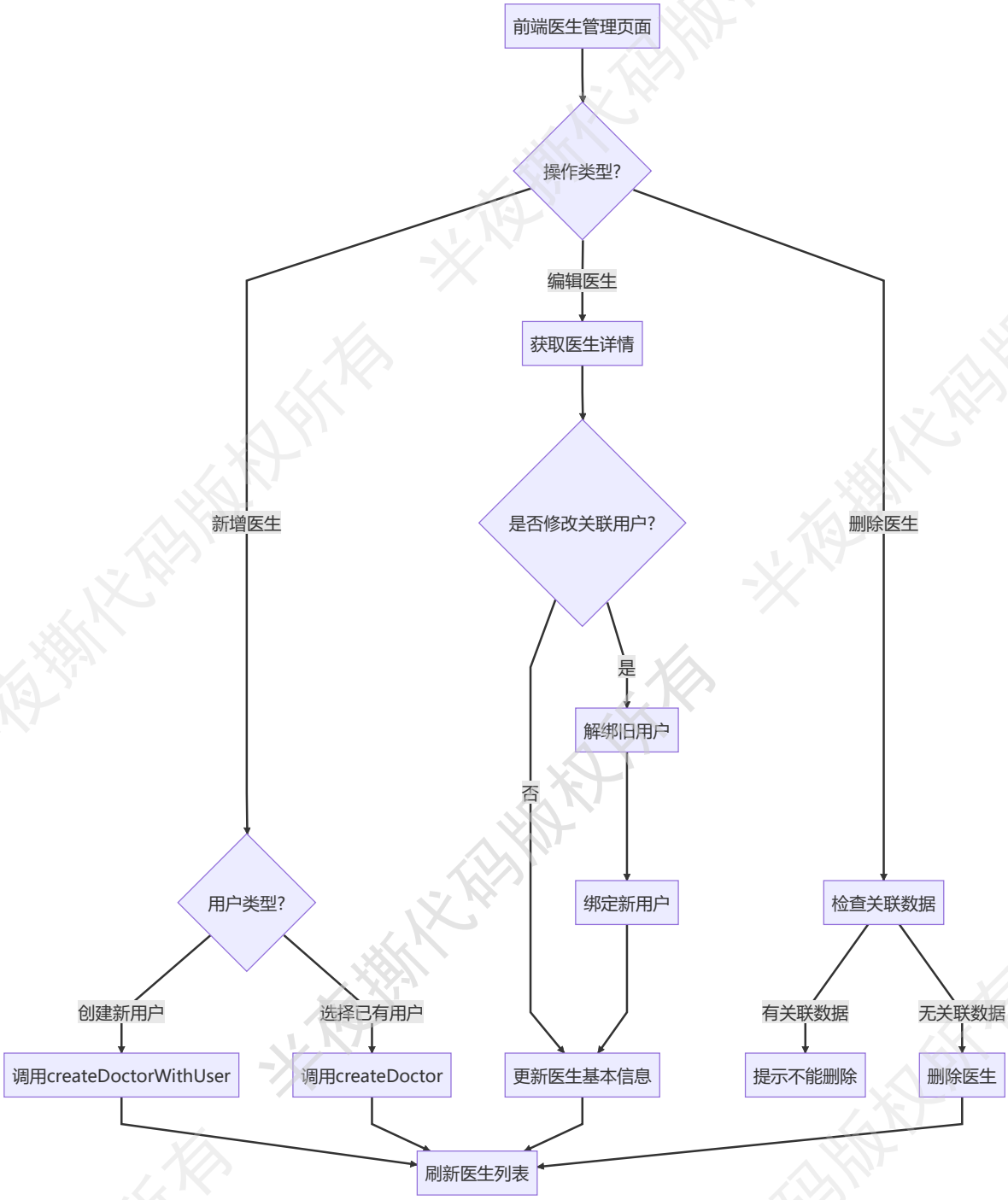
3.2 医生前台展示实现

医生前台展示模块为患者提供了浏览医生信息和预约挂号的入口。前端通过调用后端API获取医生列表和详情，支持按科室、姓名、职称等条件筛选。医生信息以卡片形式展示，包括姓名、职称、科室、专长等基本信息，点击卡片可查看详情。详情页面分为左右两部分，左侧显示医生基本信息，右侧使用选项卡展示医生简介、专业特长、出诊信息和患者评价。出诊信息以日历卡片形式展示，使用不同颜色标识排班状态，直观展示可预约时间。系统提供"预约挂号"按钮，登录用户可直接进行预约操作。整个页面采用响应式设计，适配不同设备屏幕，提升用户体验。

3.3 医生账号管理实现

医生账号管理模块实现了医生用户账号的创建和管理。系统支持两种方式创建医生：一是选择已有用户关联，二是同时创建医生和用户账号。创建用户时，系统会自动设置角色为"DOCTOR"，使医生可以登录系统进行相关操作。系统实现了用户与医生的绑定和解绑功能，严格控制一个用户只能关联一个医生，一个医生也只能关联一个用户，确保数据一致性。前端表单根据选择的用户类型动态调整验证规则，对用户名、密码、邮箱和手机号进行格式验证。编辑医生时，如果修改了关联用户，系统会自动处理解绑和绑定操作，简化用户操作流程。

半夜撕代码版权所有


前端医生管理页面
操作类型?
新增医生
编辑医生
删除医生
获取医生详情
是否修改关联用户?
是
否
解绑旧用户
绑定新用户
用户类型?
创建新用户
选择已有用户
调用createDoctorWithUser
调用createDoctor
更新医生基本信息
检查关联数据
有关联数据
无关联数据
提示不能删除
删除医生
刷新医生列表

半夜撕代码版权所有